# EDUGENIE: GOOGLE GEMINI POWERED LEARNING ASSISTANT

## Phase 5: Project Development Phase

The Project Development Phase involved implementing the AI modules, backend APIs, frontend interface, and live integration.

### 5.1 Explanation Module

EduGenie uses the **LaMini-Flan-T5-783M** model to explain complex concepts in simple and readable language.

The module is designed to provide concise and context-aware explanations, making difficult topics easier for beginners, students, and self-learners to understand.

**Fig: Explanation Module Logic**

### 5.2 Q&A Module

The Q&A Module uses **Gemini 1.5 Pro** to answer academic and general knowledge questions.

The user's question is sent to the backend, processed using Gemini, and the generated response is returned to the web interface.

**Fig: Q&A Module**

### 5.3 Quiz Module

The Quiz Module generates three multiple-choice questions from a given passage.

Each question contains:

- One question
- Four options
- One correct answer

Gemini is provided with a structured prompt to generate relevant questions and plausible distractors.

The expected response is structured in JSON format. The `clean_json_block` function removes Markdown code-block formatting before the response is parsed into a Python list.

If an error occurs during generation or parsing, an appropriate error message is returned.

**Fig: Quiz Module Logic**

## 5.4 Summary Module

The Summary Module uses Gemini to summarize long paragraphs into concise and easy-to-understand content.

It retains the important information while removing unnecessary repetition, making the content useful for quick revision.

**Fig: Summary Module**

## 5.5 Learning Path Module

The Learning Path Module uses the `get_learning_recommendations` function to generate personalized learning paths.

The generated path can include:

- Beginner-level concepts
- Intermediate concepts
- Advanced concepts
- Videos
- Articles
- Books
- Step-by-step guidance

**Fig: Learning Path Module**

## 5.6 Backend API Development

FastAPI was used to connect all the modules.

The implemented endpoints are:

| Endpoint | Function |
|---|---|
| `/qa` | Question answering |
| `/explain` | Concept explanation |
| `/quiz` | Quiz generation |
| `/summarize` | Text summarization |
| `/learn/recommendations` | Learning recommendations |

**Fig: API Endpoints**

## 5.7 Frontend Development

The web interface was developed using HTML, Jinja2, and CSS.

The interface includes:

- Task selection dropdown
- Text area for user input
- Submit button
- Result display area

The available tasks include:

- Explain
- Q&A
- Quiz
- Summary
- Recommend Path

CSS was used to provide responsive design, styled buttons, input areas, and result containers.

**Fig: Frontend Integration**

## 5.8 Live Integration

The frontend form sends a POST request to the FastAPI backend. The appropriate module processes the request and returns the result, which is displayed on the webpage.

```
User Input
 ↓
Frontend
 ↓
POST Request
```

↓
FastAPI
↓
AI Module
↓
AI Model
↓
Response
↓
Frontend Result